Jaehyeong Kim | AI Engineer / Backend Engineer

I build services that use AI to make people's lives better.

kkuldangi2@gmail.com GitHub ↗ Blog ↗ LinkedIn ↗

SUMMARY

Improving RAG evidence selection: answer accuracy 71.7% → 93.0%, evidence sufficiency 70.0% → 95.4% (100 questions × 3 runs)

Designing the agent execution environment: user-defined skills behind approval, edit success 85.6% → 88.9% (30 requests × 3 runs)

Building data pipelines: 96.1% body text and 99.3% table cells restored across 15 PDFs; drug conversion cost about 94% below the $200 estimate ($11.18 spent)

PROJECTS & EXPERIENCE

Fruition

2026.04 — Present

LLM Wiki-based AI workspace | AI SW Maestro, 17th cohort | Deployed on AWS EKS, alpha testing
Team project (3 + designer) / AI lead

How it started: a teammate's pain point with managing documents, refined through design thinking — saved documents could not be found when needed.
Team: me (all AI), 1 backend (Spring backend), 1 frontend & DevOps (frontend, MSA deployment on AWS), external designer, 3 mentors
Intellectual property: Software copyright registration completed; patent application filed with a provisional specification.

GitHub | AI repository ↗

Improving retrieval for grounded AI answers

Finds the document evidence AI answers need, and attaches none to questions the documents cannot answer.

- Evidence selection for RAG
  Problem: In a service that answers from documents, retrieval missed evidence and attached evidence to unanswerable questions, undermining trust.
  Decision: At adoption, GPT selection cost about 2.5x and took longer; tuning weights and RRF gave similar scores across settings, so I adopted Jev, which picks only from fixed options and had first been validated on request routing (77 → 81 of 98 correct).
  Solution: Two stages: BGE-M3, the best Korean semantic model in benchmarks, and BM25 for exact terms gather candidates and Jev picks the final evidence. Compared on identical candidates with blind cross-grading, then cut candidates to 300 and compared 4, 6, 8 and 10 parallel batches, choosing 8 with no accuracy difference. Later cut candidates to 200 after re-comparing on the real-world set.
  Result: On 100 questions × 3 runs that I wrote and reviewed with my team, answer accuracy 71.7% → 93.0% and evidence sufficiency 70.0% → 95.4%. Wrong answers 7.3% → 1.0% against plain RAG. Average end-to-end response 5.5s (plain RAG 4.3s).

User-defined agent Skills with approval-based execution

Recurring tasks are saved and reused as Skills, and AI-made changes apply only after the user approves them.

- User-defined task instructions (Skills)
  Problem: Recurring tasks differ by user, so they cannot all be built as features, and AI edits arrived with parts of the request missing, leaving repeatable work to chance
  Decision: AI drafts instructions, code checks permissions and scope, and the user decides publishing and applying. Code format checks missed omitted requests, so I added LLM semantic evaluation
  Solution: Authoring, review and publishing of natural-language instructions with checks against bypassing permission or approval; preview and separate approval before changes; semantic evaluation with at most one rewrite. Generation, evaluation and rewrite run as LangGraph nodes for step-level tracing, and published Skills are stored as immutable versions in PostgreSQL
  Result: On 30 requests × 3 runs that I wrote and reviewed with my team, edit success 77 → 80 of 90 (85.6% → 88.9%); documents stayed unchanged before approval

PDF → Markdown: 96.1% body text, 99.3% table cells restored

Converted uploaded PDFs into Markdown that keeps tables and equations, so they can be searched and cited in answers.

- PDF → Markdown conversion
  Problem: Broken tables and equations in paper PDFs made the figures behind Wiki pages and answers unusable
  Decision: Repairing only detected damage missed errors, and full AI conversion lacked table/equation accuracy and was costly, so I split processing by content type. Body text uses fast AnyDoc extraction; repair moved from an sLLM to commercial LLM APIs to avoid GPU server costs
  Solution: Extracted body text with AnyDoc and repaired only garbled parts with AI, read table/equation regions separately, and kept original figures
  Result: In the adoption experiment (4 PDFs), restoration rose from about 45% with Docling alone to about 90%. Across 15 PDFs by type × 3 runs: headings 97.6%, body text 96.1%, table cells 99.3%, equations 100%, images 91.3% (AI-judged)

MSA: Splitting only by feature would keep DB load the same while adding hops, so I proposed splitting by data ownership and co-designed boundaries and communication with teammates and mentors.

All figures are from internal evaluations.

See the detailed portfolio for the problem-solving process and experimental results.

RAG evidence selection ↗ Agent execution environment ↗ PDF conversion ↗

Pilltip

2025.03 — 2025.12

Personalized AI medication service | Pusan National University capstone
Team project (3 + designer) / Core backend, data preparation

How it started: drug information is scattered, making it hard to check whether a medicine is safe for your current health condition.
Team: me (backend & AI), 1 Android (Kotlin app), 1 backend & security (user DB, login, questionnaires, AES-GCM), designer. The features below are my work.

GitHub repository ↗

Drug data preparation pipeline

Rewrote about 44,000 drug records in plain language, converting each repeated sentence once and reusing it per drug.

- **Problem:** Rewriting about 44,000 drug records as-is was estimated at about $200 in API costs, more than the budget allowed for covering every drug.
  Decision: Instead of covering fewer drugs, convert each repeated sentence once. The job needed no real-time response, so Batch API traded waiting time for lower cost.
  Solution: Split text into sentence blocks, merged exact duplicates, converted via GPT Batch API and mapped results back to each drug. Reproduced in 2026 with PySpark, which parallelizes across cores and scales out, to measure each step.
  Result: Actual API spending was $11.18 versus about $200 estimated (about 94% below the estimate); the 2025 dataset was reduced from about 1GB to 326MB. In the reproduction, exact deduplication cut text from 744MB to 44.1MB (−94.1%).

Estimated cost versus actual spending; preprocessing labor is excluded.

AI medication chatbot using health profiles

Found drug and medication information from natural-language symptoms. Evaluated health profiles internally, separately from information sent to external AI.

- **Problem:** Users checking whether a drug is safe describe symptoms in different words from drug indications, and the health data needed for personalization could not go to external AI.
  Decision: A synonym dictionary had too many overlapping symptoms, so I used semantic search; filtering the profile had too many cases, so profile decisions stay internal. Weaviate was chosen as lighter than Milvus, and Spring AI ran orchestration on the Spring backend to avoid a separate Python container.
  Solution: Weaviate semantic search and AI orchestration route to drug search, medication risk and dosage paths; pregnancy, current medications and underlying conditions are judged in internal code.
  Result: Personalized symptom-based search with those profile fields excluded from external LLM inputs.

See the detailed portfolio for the problem-solving process and experimental results.

Data pipeline case study ↗ Personalization case study ↗

Skills

AI & Search	LangChain, LangGraph, Spring AI, RAG, LLM Evaluation, Prompt injection defense LangSmith, Elasticsearch, Weaviate
Backend	Java, Spring Boot, Python
Data & Infra	MySQL, PostgreSQL, Redis, Kafka Docker, Docker Compose, GitHub Actions

Activities & contributions

2026.06 — 2026.06

Rhwp open-source contributions

Gave Claude Code a work plan and contribution rules to automate the flow from issue analysis to PR, including HWP/HWPX save-error fixes; 7 PRs merged. Listed as a contributor.

GitHub repository ↗ PR #1213 | Merged ↗ PR #1351 | Submitted ↗

2025.03 — 2026.01



APPTIVE, Pusan National University

Backend mentee and mentor. Led 6 mentoring and code-review sessions for 12 mentees. Helped revise the curriculum to strengthen HTTP, Servlet, REST API and database foundations.

APPTIVE Backend Mentor Merit Award | 2026.01.09

2025.11 / 2025.07



SK AI SUMMIT, K-ICT WEEK in Busan

Represented Pusan National University at the Pilltip booth, demonstrating the service to more than 100 visitors and answering technical questions. Most of the 10 pieces of feedback asked for health supplement coverage.

2022.02 — 2025.07



945-day Baekjoon problem-solving streak

Aiming for one problem a day, I reached a longest streak of 945 consecutive days on solved.ac. 1,659 problems solved, solved.ac Platinum IV. Solutions are recorded in a GitHub repository.

Baekjoon solutions repository ↗

Education & certification

2022.02 — 2026.02 (Transfer)

Pusan National University

GPA 4.12 / 4.5

2019.03 — 2022.02 (Withdrew)

Daegu University

GPA 4.2 / 4.5

2025.09

Engineer Information Processing

Human Resources Development Service of Korea

Awards

2025.10



Capstone Design Gold Award
Software & AI division / College of Information and Biomedical Engineering, Pusan National University

2025.09



Busan DATA WEEK Top Excellence Award
Data utilization case competition / Busan Techno Park

2025.09



AI LAUNCH Career School Startup Hackathon Encouragement Award
KRYPTON X x Root Impact x Google.org

2025.08



SW University Digital Competition Sponsor Award
SW University Council